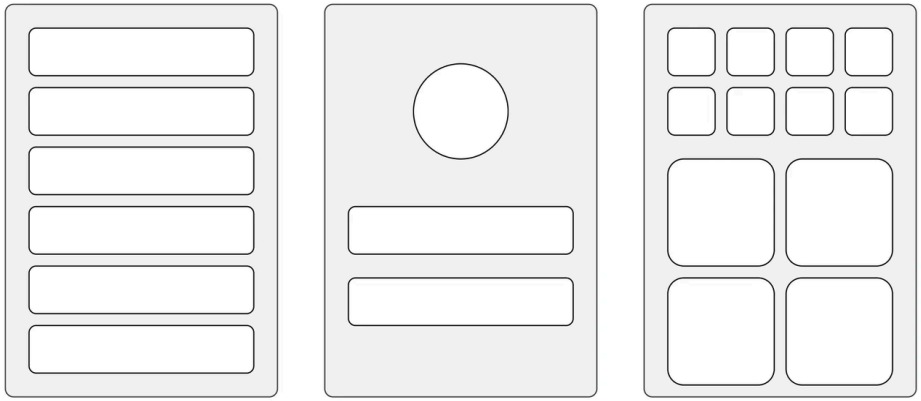
常用布局

1. 布局基础

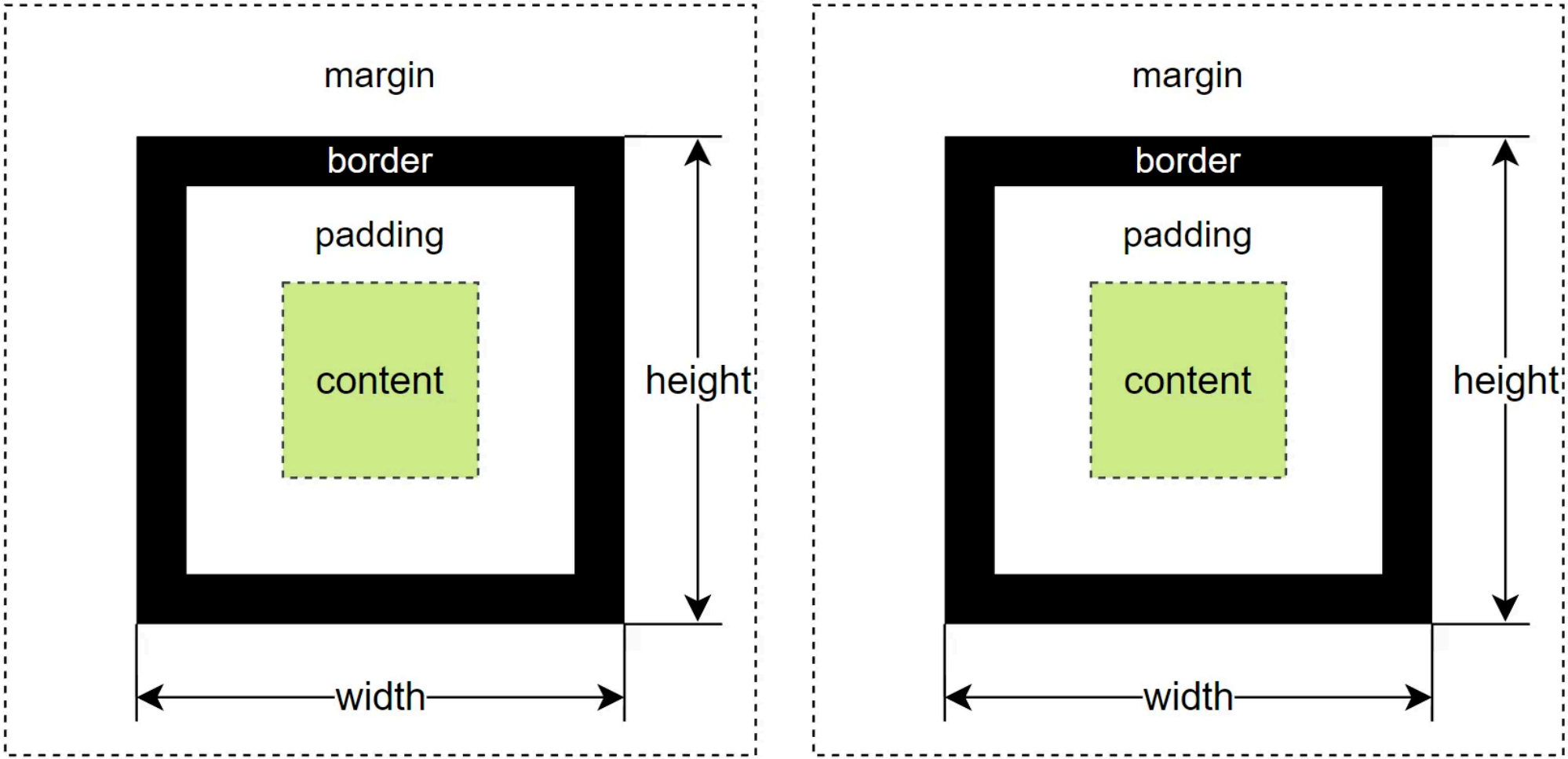
1.1.1 概述

布局是指对页面组件进行排列和定位的过程，其目的是有效地组织和展示页面内容，会涉及到组件的大小、位置以及它们之间的相互关系等等。



1.1.2 盒子模型

在鸿蒙应用中，页面上的每个组件都可以看做是一个矩形的盒子，这个盒子包含了**内容区域**（content）、**边框**（border）、**内边距**（padding）和**外边距**（margin），各部分内容如下图所示



其中**margin**、**padding**和**border**均可使用同名的属性方法进行设置，各方法定义如下

- **margin**

代码块

```
margin(value: { top?:Length, right?:Length, bottom?:Length, left?:Length } |
 Length )
```

> ❗ **说明：**
>
> 1. Length = string | number | Resource
> 2. 当参数类型为Length时，四个方向的边距同时生效

- **padding**

```
代码块

1   padding(value: { top?:Length, right?:Length, bottom?:Length, left?:Length }
    |  Length )
```

- **border**

```
代码块

1   border(value: {width?:Length, color?:ResourceColor, radius?:Length,
    style?:BorderStyle })
```

  - **width**

`width` 属性表示边框宽度

  - **color**

`color` 属性表示边框颜色

  - **radius**

`radius` 属性表示边框圆角半径

  - **style**

`style` 属性表示边框样式，可通过 `BorderStyle` 这一枚举类型进行设置，可选的枚举值有

名称	描述	效果

BorderStyle.Dotted	显示为一系列圆点	
BorderStyle.Dashed	显示为一系列短的方形虚线	
BorderStyle.Solid	显示为一条实线	

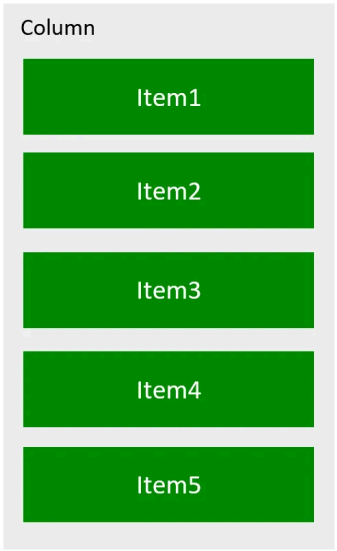
代码块

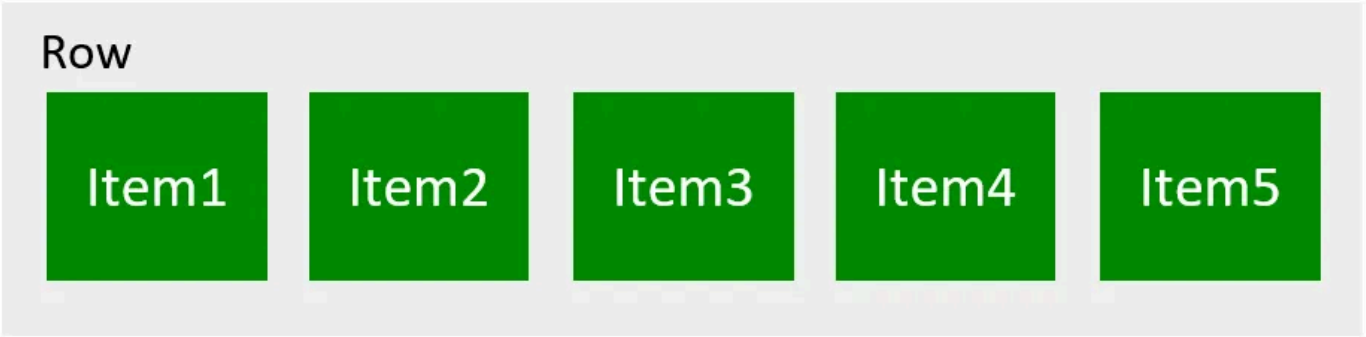
```
@Entry
@Component
struct comLayout {
  build() {
    Text('这是一行文本')
      .fontSize(30)
      .padding(30)
      .margin(20)
      .border({
        width: 5,
        color: Color.Black,
        radius: 10,
        style: BorderStyle.Dotted
      })
      .borderWidth(1)
  }
}
```

2. 线性布局（Column/Row）

2.1 概述

线性布局（`LinearLayout`）是开发中最常用的布局，可通过容器组件`Column`和`Row`构建，其子组件会在垂直或者水平方向上进行线性排列，具体效果如下图所示



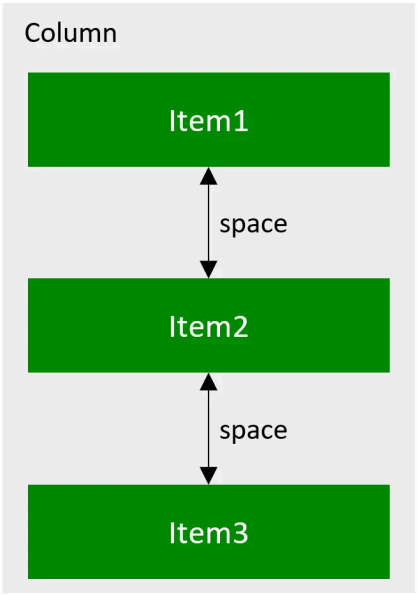


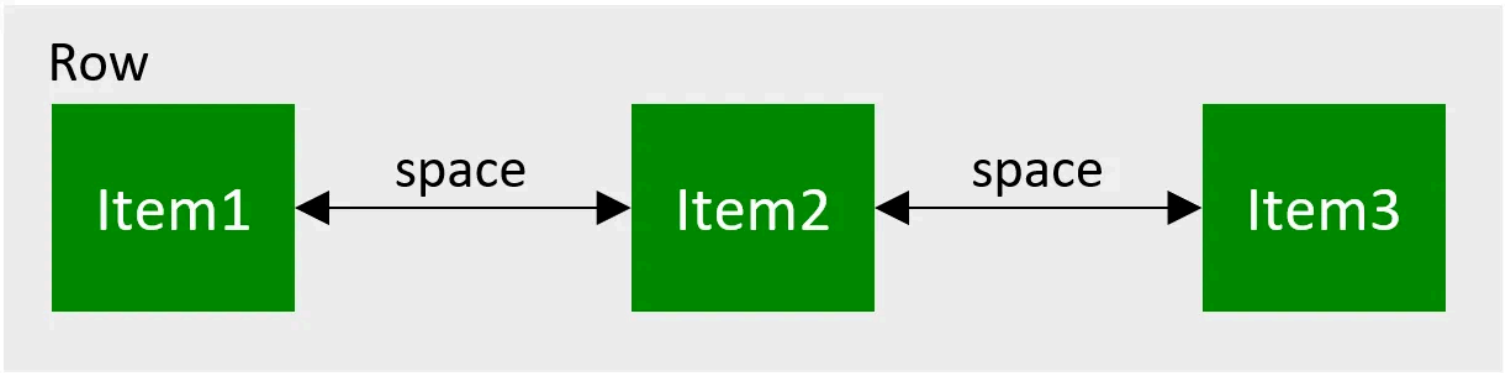
> **！ 说明**
>
> **Column**和**Row**容器均有两个轴线，分别是**主轴**和**交叉轴**
>
> **主轴**：线性布局容器在布局方向上的轴线，**Row**容器**主轴为横向**，**Column**容器**主轴为纵向。**

交叉轴：垂直于主轴方向的轴线。**Row容器交叉轴为纵向**，**Column容器交叉轴为横向。**

2.2 参数

Column和Row容器的参数类型为 `{space?: string | number}` ，开发者可通过 `space` 属性调整子元素在**主轴方向**上的间距，效果如下





代码块

```
@Entry
@Component
struct SpacePage {
  build() {
    Column({ space: 50 }) {
      Row()
        .backgroundColor(Color.Green)
        .width('80%')
        .height(70)

      Row()
        .backgroundColor(Color.Green)
        .width('80%')
        .height(70)

      Row()
        .backgroundColor(Color.Green)
        .width('80%')
        .height(70)

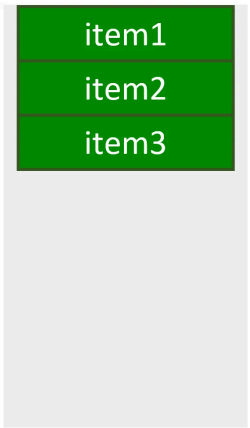
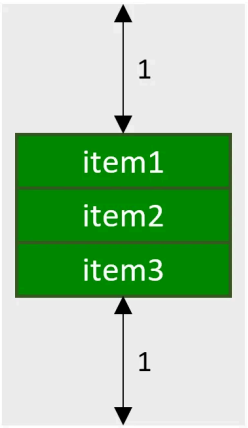
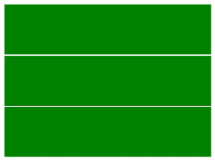
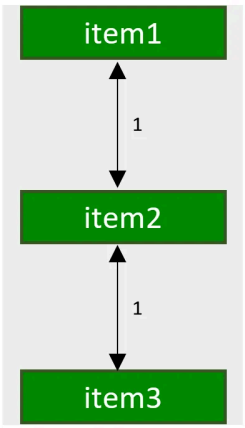
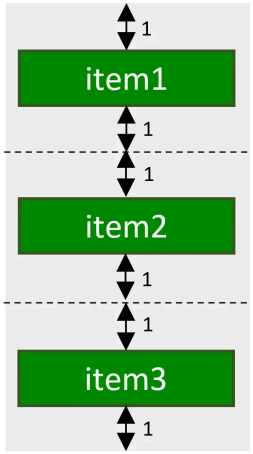
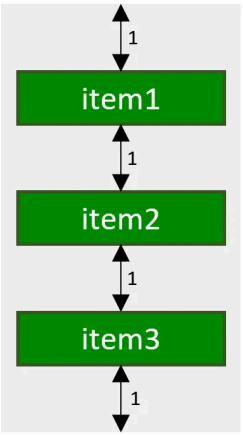
    }.width('100%')
    .height('100%')
    .backgroundColor('#eeeeee')
  }
}
```

2.3 常用属性

2.3.1 子元素沿主轴方向的排列方式 `justifyContent()`

子元素沿主轴方向的排列方式可以通过 `justifyContent()` 方法进行设置，其参数类型为枚举类型 `FlexAlign` ，可选的枚举值有

名称	Start	Center	End	SpaceBetween	SpaceAround	SpaceEvenly
描述	头部对齐	居中对齐	尾部对齐			

				均匀分布，首尾两项两端对齐，中间元素等间距分布	均匀分布，所有子元素两侧都留有相同的空间	均匀分布，所有子元素之间以及首尾两元素到两端的距离都相等
效果(以Column容器为例)						

代码块

```
@Entry
@Component
struct AlignItemsPage {
  build() {
    Column({ space: 2 }) {
      Row()
        .backgroundColor(Color.Green)
        .width('80%')
        .height(70)
      Row()
        .backgroundColor(Color.Green)
        .width('80%')
        .height(70)
      Row()
```

```
15            .backgroundColor(Color.Green)
16            .width('80%')
17            .height(70)
18        }.width('100%')
19        .height('100%')
20        .justifyContent(FlexAlign.Center) // 设置子元素沿主轴方向的排列方式
21      }
22    }
```

2.3.2 子元素沿交叉轴方向的对齐方式 `alignItems()`

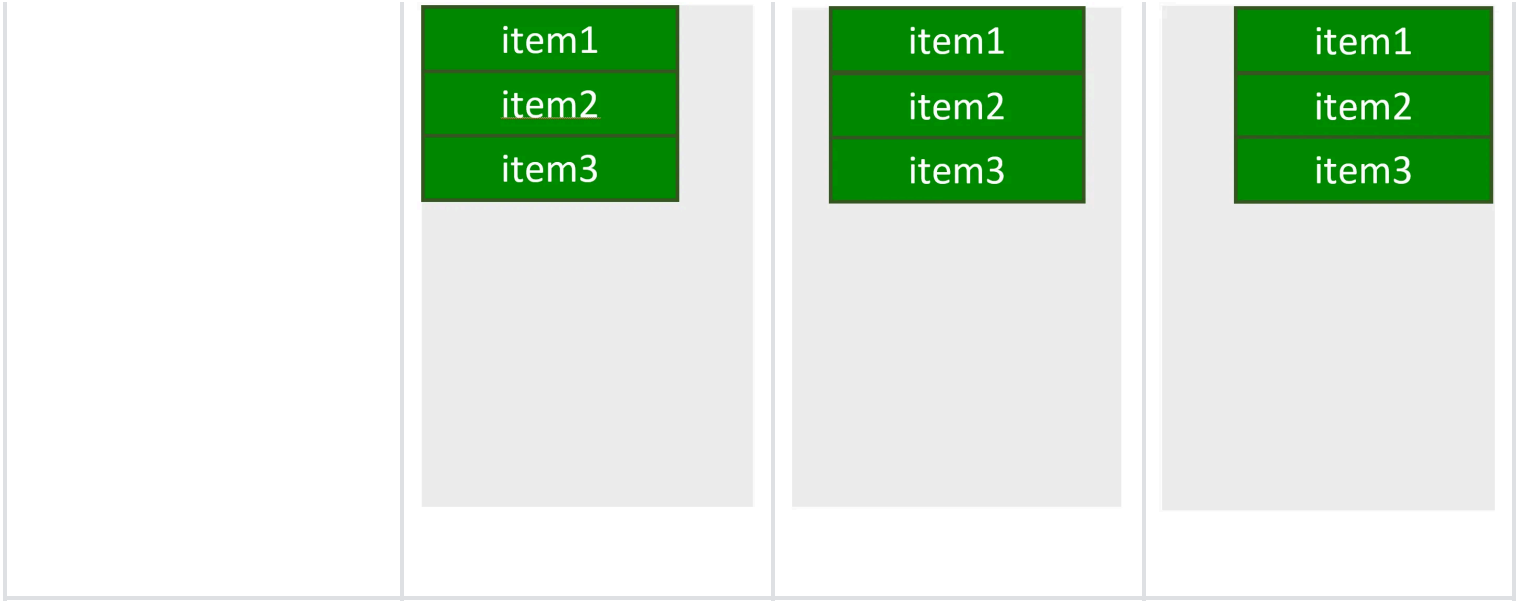
子元素沿交叉轴方向的对齐方式可以通过 `alignItems()` 方法进行设置，

其参数类型，

- 对于 `Column` 容器来讲是水平对齐 `HorizontalAlign` ，
- 对于 `Row` 容器来讲是垂直对齐 `VerticalAlign` ，

两者都是枚举类型，可选择枚举值也都相同，具体内容如下

名称	Start	Center	End
描述	头部对齐	居中对齐	尾部对齐
效果 (以Column容器为例)			



代码块

```
@Entry
@Component
struct AlignItemsPage {
  build() {
    Column({ space: 2 }) {
      Row()
        .backgroundColor(Color.Green)
        .width('80%')
        .height(70)
      Row()
        .backgroundColor(Color.Green)
        .width('80%')
        .height(70)
      Row()
        .backgroundColor(Color.Green)
        .width('80%')
```

```
17            .height(70)
18        }.width('100%')
19        .height('100%')
20        .alignItems(HorizontalAlign.Start)
21      }
22    }
```

2.4 实用技巧

2.4.1 Blank 组件的使用

Blank可作为**Column**和**Row**容器的子组件，该组件不显示任何内容，并且会**始终充满容器主轴方向上的剩余空间**，效果如下：



代码块

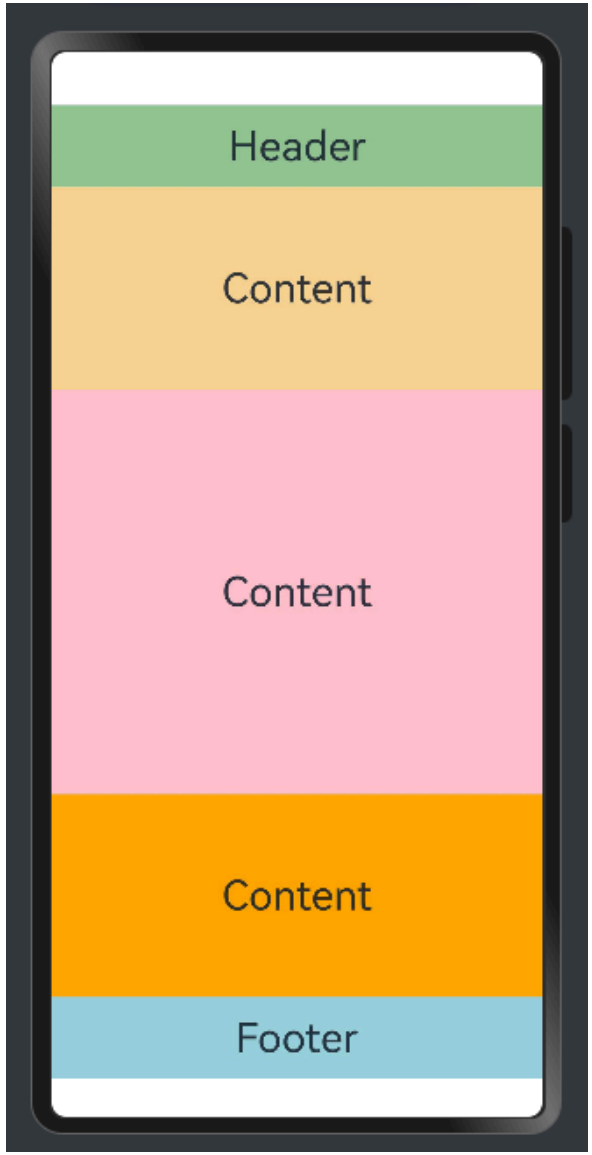
```
1   @Entry
2   @Component
3   struct BlankPage {
4     build() {
5       Column({ space: 50 }) {
6         Row() {
7           Image($r('app.media.icon_bluetooth'))
8             .width(30)
9             .height(30)
10          Text('蓝牙')
11            .fontSize(20)
12            .margin({ left: 5 })
13          Blank()
14          Toggle({ type: ToggleType.Switch })
15        }
```

```
        .width('90%')
        .height(60)
        .backgroundColor(Color.White)
        .padding({ left: 10, right: 10 })
        .borderRadius(15)
        .shadow({ radius: 10 })
      }.width('100%')
      .height('100%')
      .justifyContent(FlexAlign.Center)
      .backgroundColor('#f2f2f2')
    }
  }
```

2.4.2 layoutWeight 属性的使用

`layoutWeight` 属性可用于**Column**和**Row**容器的子组件，其作用是配置子组件**在主轴方向上的尺寸权重。**

配置该属性后，子组件沿主轴方向的尺寸设置（ `width` 或 `height` ）将失效，具体尺寸根据权重进行计算，计算规则如下图所示：



代码块

```
@Entry
@Component
struct LayoutWeightPage {
  build() {
```

```
    Column() {
      //Header:高度60vp
      Row() {
        Text('Header')
          .fontSize(30)
      }.backgroundColor('#66008000')
      .justifyContent(FlexAlign.Center)
      .height(60)
      .width('100%')
      //Content:充满剩余空间
      // Footer:高度60vp
      Row() {
        Text('Content')
          .fontSize(30)
      }.backgroundColor('#66ffa200')
      .justifyContent(FlexAlign.Center)
      .width('100%')
      .layoutWeight(1)

      Row() {
        Text('Content')
          .fontSize(30)
      }.backgroundColor(Color.Pink)
      .justifyContent(FlexAlign.Center)
      .width('100%')
      .layoutWeight(2)

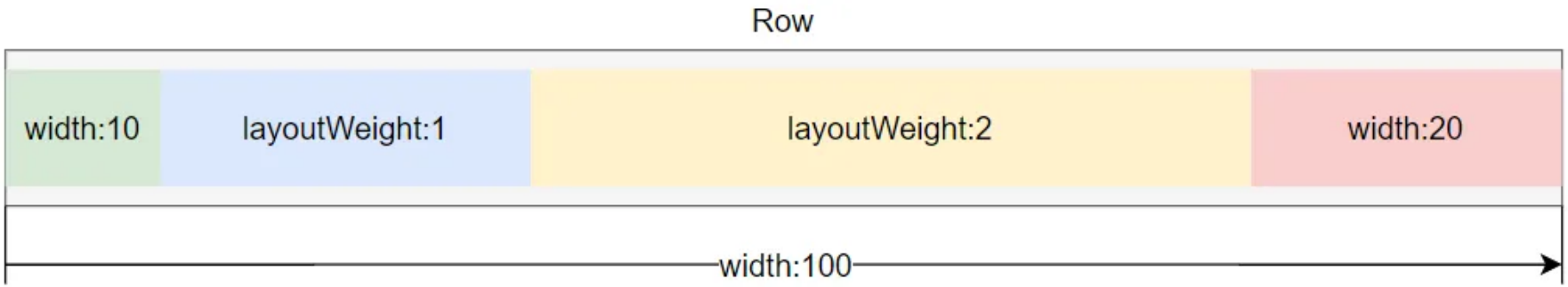
      Row() {
        Text('Content')
          .fontSize(30)
      }.backgroundColor(Color.Orange)
      .justifyContent(FlexAlign.Center)
      .width('100%')
      .layoutWeight(1)

      Row() {
        Text('Footer')
```

```
            .fontSize(30)
        }.backgroundColor('#660e9fba')
        .justifyContent(FlexAlign.Center)
        .width('100%')
        .height(60)

      }.width('100%')
      .height('100%')
      .justifyContent(FlexAlign.Center)
      .backgroundColor('#f2f2f2')
    }
  }
```

练习：



代码块

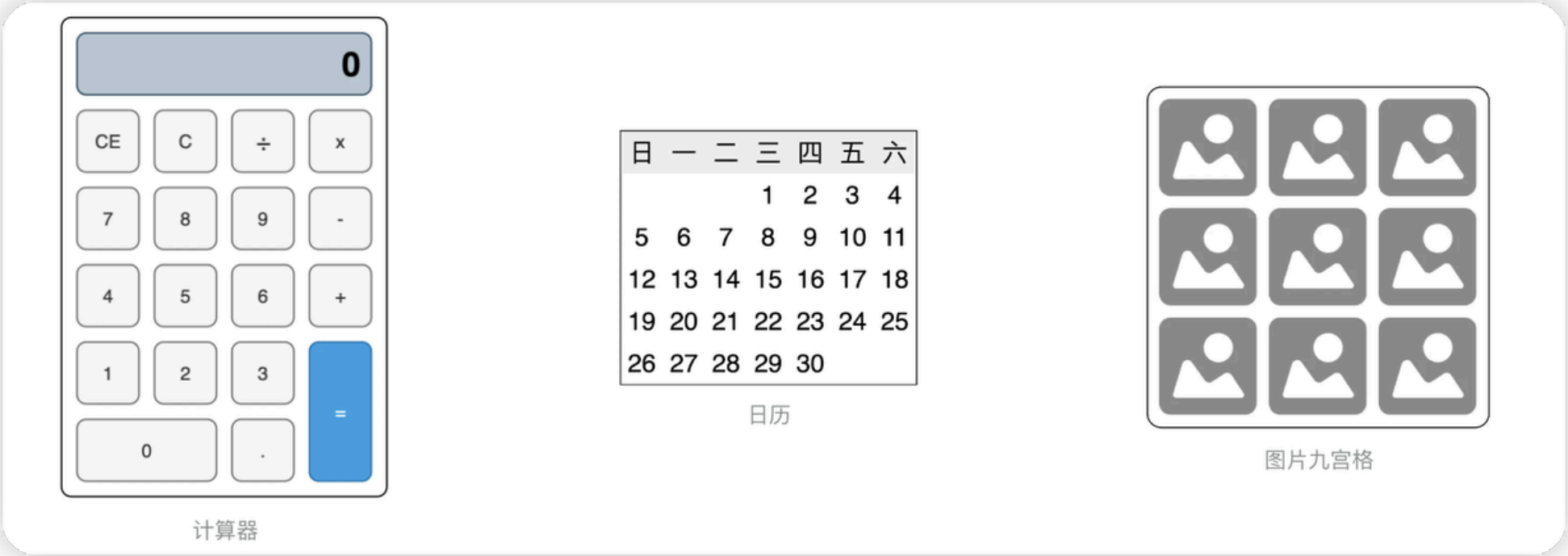
```
@Entry
@Component
struct LayoutWeightPage {
  build() {
    Row(){
      Row(){
        Text('width:10')
          .width('100%')
      }.width(10).backgroundColor(Color.Green).height(50)
      Row(){
```

```
          Text('LayoutWeight:1')
            .width('100%')
        }.layoutWeight(1).backgroundColor(Color.Red).height(50)
        Row(){
          Text('LayoutWeight:2')
            .width('100%')
        }.layoutWeight(2).backgroundColor(Color.Blue).height(50)
        Row(){
          Text('width:10')
            .width('100%')
        }.width(20).backgroundColor(Color.Green)
      }.width('100%')
      .height(50)
      .justifyContent(FlexAlign.Center)
    }
  }
```

3. 网格布局（Grid）

3.1 概述

网格布局（`Grid`）是一种强大的布局方案，它将页面划分为**行**和**列**组成的网格，然后将页面内容在二维网格中进行自由的定位，以下效果都可通过网格布局实现



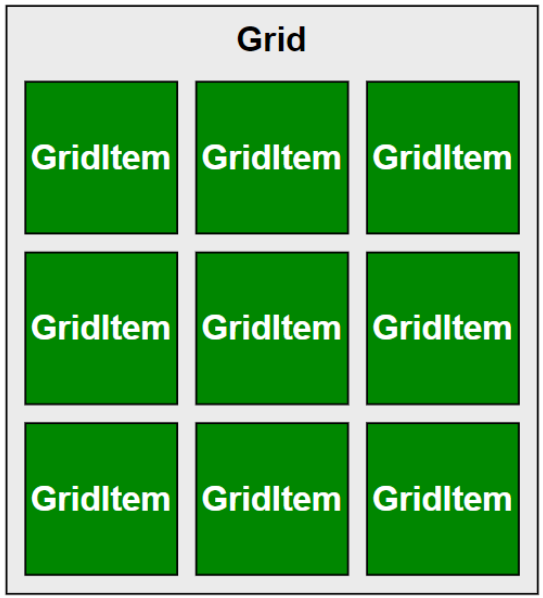
网格布局的容器组件为 **Grid**，子组件为 **GridItem**，具体语法如下

代码	效果

代码块

```
Grid() {
  GridItem() {
    Text('GridItem')
  }
  GridItem() {
    Text('GridItem')
  }
  GridItem() {
    Text('GridItem')
```

```
    }
    GridItem() {
      Text('GridItem')
    }
    ......
  }
```



3.2 常用属性

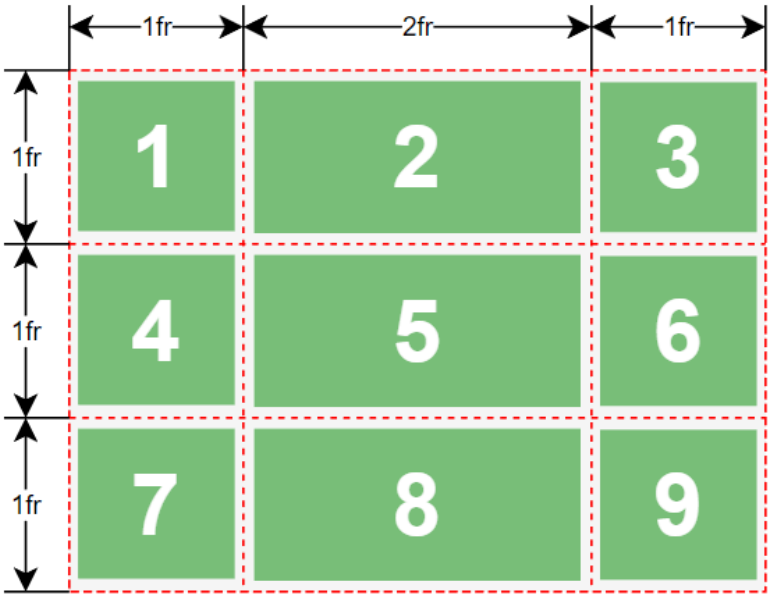
3.2.1 划分网格

Grid组件支持**自定义行数**和**列数**以及每行和每列的尺寸大小，上述内容需要使用

`rowsTemplate()` 方法和 `columnsTemplate()` 方法进行设置，具体用法如下

代码	效果
代码块 `Grid() { ForEach([1, 2, 3, 4, 5, 6, 7, 8, 9], (item) => { GridItem() { Text(item.toString()) .itemTextStyle() } }) `	

```
  }
  .width(320)
  .height(240)
  .rowsTemplate('1fr 1fr 1fr')
  .columnsTemplate('1fr 2fr 1fr')
```



！ 说明：

fr为 fraction(比例、分数) 的缩写。fr的个数表示网格布局的行数或列数，fr前面的数值大小，表示该行或列的尺寸占比。

代码块

```
@Entry
@Component
struct GridBasic {
  build() {
    Column() {
      Grid() {
        ForEach([1, 2, 3, 4, 5, 6, 7, 8, 9], (item:number):void => {
          GridItem() {
            Text(item.toString())
              .itemTextStyle()
```

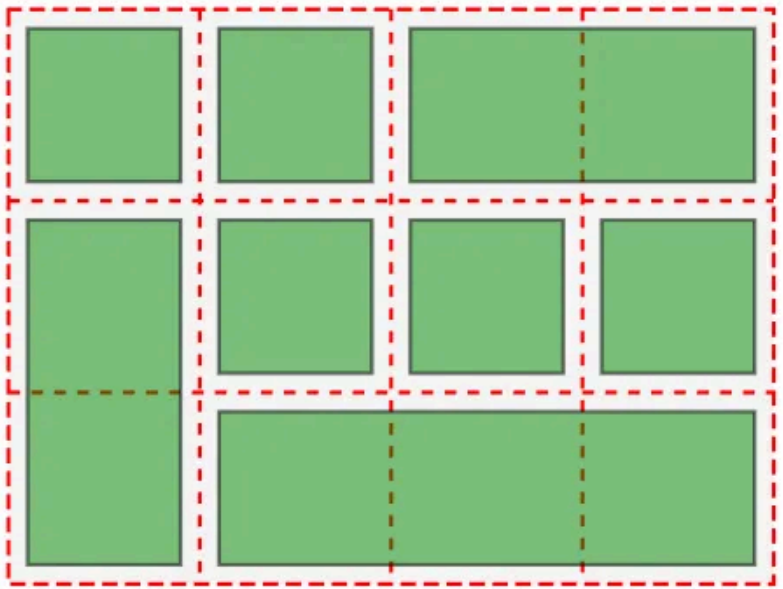
```
            }
          })
        }
        .width(320)
        .height(240)
        .rowsTemplate('1fr 1fr 1fr')
        .columnsTemplate('1fr 2fr 1fr')
        .gridStyle()
      }.width('100%')
      .height('100%')
      .justifyContent(FlexAlign.Center)
    }
  }

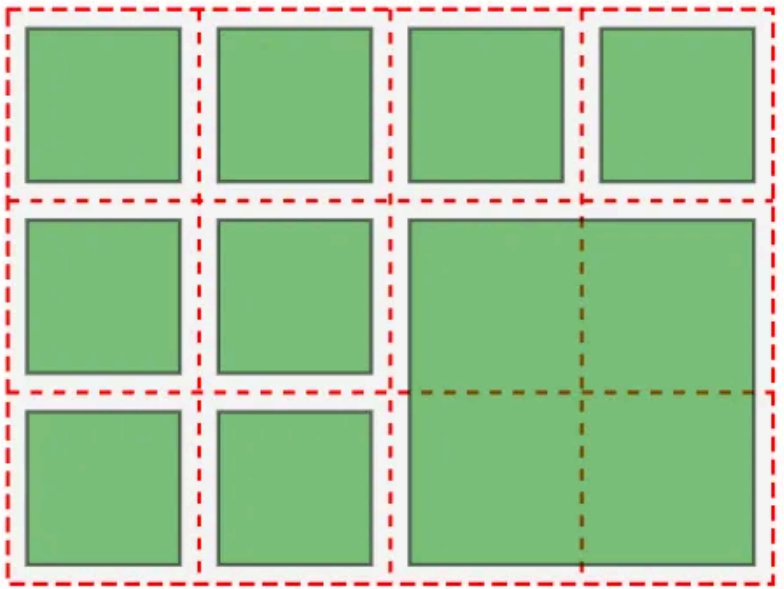
  @Extend(Text) function itemTextStyle() {
    .height('100%')
    .width('100%')
    .textAlign(TextAlign.Center)
    .fontColor(Color.White)
    .fontSize(40)
    .fontWeight(FontWeight.Bold)
    .backgroundColor('#008a00')
    .borderWidth(1)
  }

  @Extend(Grid) function gridStyle() {
    .backgroundColor('#f5f5f5')
    .borderWidth(1)
  }
```

3.2.2 子组件所占行列数s

GridItem组件支持横跨几行或者几列，如下图所示





可以使用 `columnStart()` 、 `columnEnd()` 、 `rowStart()` 和 `rowEnd()` 方法设置 **GridItem** 组件所占的单元格，

- `rowStart` 和 `rowEnd` 属性表示当前子组件的起始**行号**和终点**行号**，
- `columnStart` 和 `columnEnd` 属性表示指定当前子组件的起始**列号**和终点**列号**。

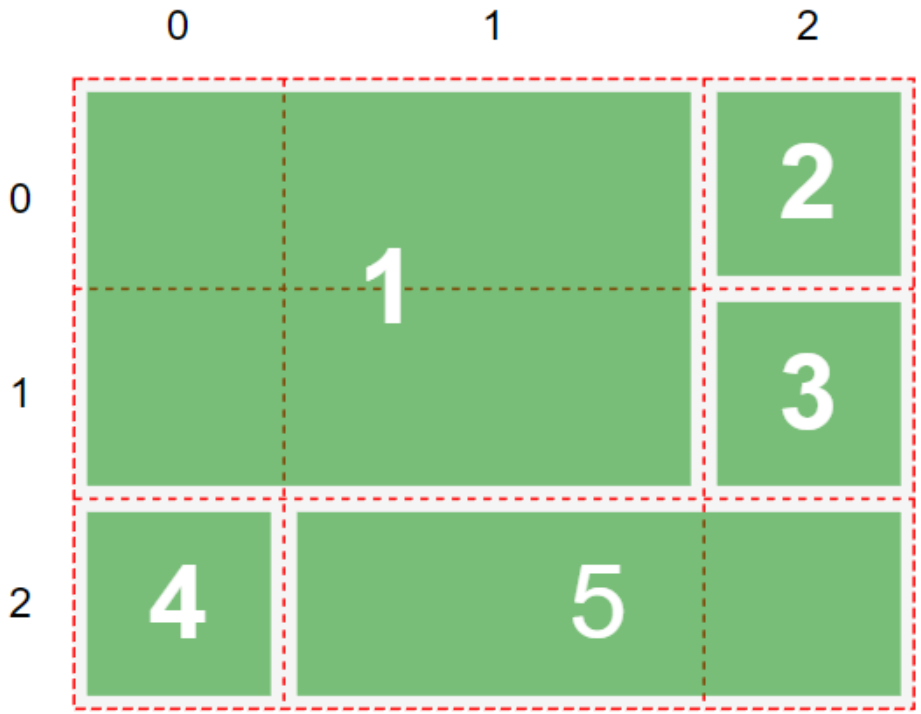
> **！ 说明：**
>
> **Grid**容器中的行号和列号均从**0**开始。

具体用法如下

代码	效果

代码块
1 Grid() {
2 GridItem() {
3 Text('1')	

```
4         .itemTextStyle()
5     }
6     .rowStart(0)
7     .rowEnd(1)
8     .columnStart(0)
9     .columnEnd(1)
10
11    GridItem() {
12      Text('2')
13        .itemTextStyle()
14    }
15
16    GridItem() {
17      Text('3')
18        .itemTextStyle()
19    }
20
21    GridItem() {
22      Text('4')
23        .itemTextStyle()
24    }
25    GridItem() {
26      Text('5')
27        .itemTextStyle()
28    }
29    .columnStart(1)
30    .columnEnd(2)
31  }
32  .width(320)
33  .height(240)
34  .rowsTemplate('1fr 1fr 1fr')
35  .columnsTemplate('1fr 2fr 1fr')
36  .gridStyle()
```



代码块

```
@Entry
@Component
struct StartAndEndPage {
  build() {
    Column() {
      Grid() {
        GridItem() {
          Text('1')
            .itemTextStyle()
        }.rowStart(0).rowEnd(1).columnStart(0).columnEnd(1)

        GridItem() {
          Text('2')
            .itemTextStyle()
        }

        GridItem() {
          Text('3')
            .itemTextStyle()
        }

        GridItem() {
          Text('4')
            .itemTextStyle()
        }

        GridItem() {
          Text('5')
            .itemTextStyle()
        }.columnStart(1).columnEnd(2)
      }
      .width(320)
      .height(240)
      .rowsTemplate('1fr 1fr 1fr')
      .columnsTemplate('1fr 2fr 1fr')
      .gridStyle()
    }.width('100%')
```

```
38        .height('100%')
39        .justifyContent(FlexAlign.Center)
40      }
41    }
42
43    @Extend(Text) function itemTextStyle() {
44      .height('100%')
45      .width('100%')
46      .textAlign(TextAlign.Center)
47      .fontColor(Color.White)
48      .fontSize(40)
49      .fontWeight(FontWeight.Bold)
50      .backgroundColor('#008a00')
51      .borderWidth(1)
52    }
53
54    @Extend(Grid) function gridStyle() {
55      .backgroundColor('#f5f5f5')
56      .borderWidth(1)
57    }
```

🧐 练习:

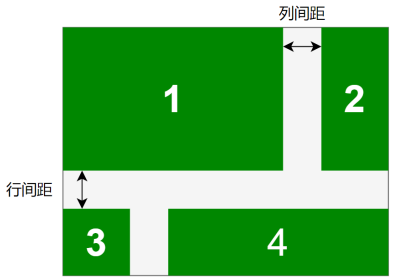
完成前面两个页面

3.2.3 行列间距

使用 `rowsGap()` 和 `columnsGap()` 属性，可以控制行列间距，具体用法如下

代码	效果
代码块 1 Grid() {	

```
    ......
  }
  .columnsGap(20)
  .rowsGap(20)
```



代码块

```
@Entry
@Component
struct GridGap {
  build() {
    Column() {
      Grid() {
        GridItem() {
          Text('1')
            .itemTextStyle()
        }.rowStart(0).rowEnd(1).columnStart(0).columnEnd(1)

        GridItem() {
          Text('2')
            .itemTextStyle()
        }.rowStart(0).rowEnd(1)


        GridItem() {
          Text('3')
            .itemTextStyle()
        }

        GridItem() {
          Text('4')
            .itemTextStyle()
        }.columnStart(1).columnEnd(2)
```

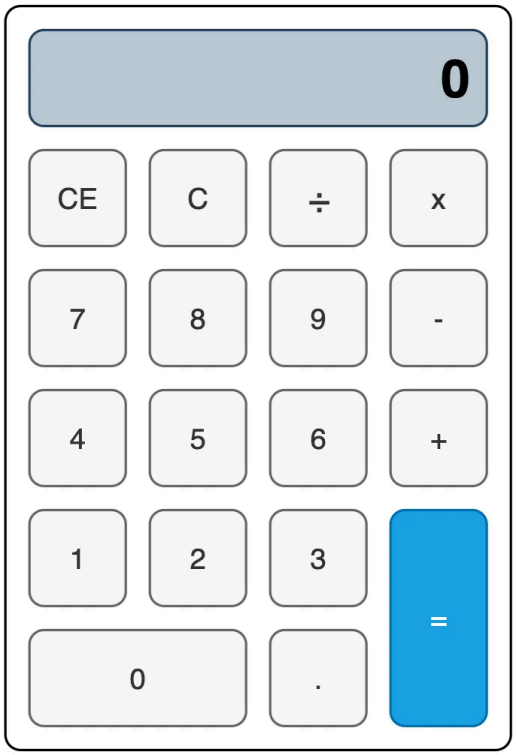
```
        }
        .width(320)
        .height(240)
        .rowsTemplate('1fr 1fr 1fr')
        .columnsTemplate('1fr 2fr 1fr')
        .gridStyle()
        .rowsGap(20)
        .columnsGap(20)
      }.width('100%')
      .height('100%')
      .justifyContent(FlexAlign.Center)
    }
  }

  @Extend(Text) function itemTextStyle() {
    .height('100%')
    .width('100%')
    .textAlign(TextAlign.Center)
    .fontColor(Color.White)
    .fontSize(40)
    .fontWeight(FontWeight.Bold)
    .backgroundColor('#008a00')
    .borderWidth(1)
  }

  @Extend(Grid) function gridStyle() {
    .backgroundColor('#f5f5f5')
    .borderWidth(1)
  }
```

3.2.4 网格布局Grid 课后练习

使用网格布局实现如下布局效果



代码块

```
@Entry
@Component
struct CalculatorPage {
  build() {
    Column() {
      Grid() {
        GridItem() {
          Text('0')
            .screenTextStyle()
        }.columnStart(0).columnEnd(3)

        GridItem() {
          Text('CE')
            .buttonTextStyle()
        }
```

```
16
17          GridItem() {
18            Text('C')
19              .buttonTextStyle()
20          }
21
22          GridItem() {
23            Text('÷')
24              .buttonTextStyle()
25          }
26
27          GridItem() {
28            Text('x')
29              .buttonTextStyle()
30          }
31
32          GridItem() {
33            Text('7')
34              .buttonTextStyle()
35          }
36
37          GridItem() {
38            Text('8')
39              .buttonTextStyle()
40          }
41
42          GridItem() {
43            Text('9')
44              .buttonTextStyle()
45          }
46
47          GridItem() {
48            Text('-')
49              .buttonTextStyle()
50          }
51
52          GridItem() {
```

```
53            Text('4')
54              .buttonTextStyle()
55          }
56
57          GridItem() {
58            Text('5')
59              .buttonTextStyle()
60          }
61
62          GridItem() {
63            Text('6')
64              .buttonTextStyle()
65          }
66
67          GridItem() {
68            Text('+')
69              .buttonTextStyle()
70          }
71
72          GridItem() {
73            Text('1')
74              .buttonTextStyle()
75          }
76
77          GridItem() {
78            Text('2')
79              .buttonTextStyle()
80          }
81
82          GridItem() {
83            Text('3')
84              .buttonTextStyle()
85          }
86
87          GridItem() {
88            Text('=')
89              .buttonTextStyle()
```

4. 列表布局（List）

4.1 概述

List是一个功能强大的容器组件，使用**List**可以轻松高效地显示结构化、可滚动的列表信息，例如通讯录、新闻列表等等。





List容器的子组件为**ListItem**或者**ListItemGroup**，其中，

ListItem表示单个列表项，

ListItemGroup用于列表数据的分组展示，其子组件也是**ListItem**，

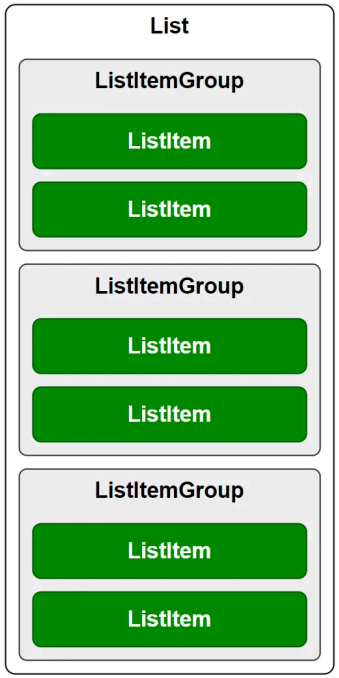
具体用法如下

代码	效果

代码块
``
List() {
  // 列表项
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
  ListItem() {.....}
}
``
	

代码块
``
List() {
  // 列表组
  ListItemGroup(){
    //列表项
    ListItem(){.....}
    ListItem(){.....}
  }
 	
  ListItemGroup(){
    ListItem(){.....}
    ListItem(){.....}
  }
 	
  ListItemGroup(){
``	

```
15        ListItem(){......}
16        ListItem(){......}
17      }
18    }
```



4.2 参数

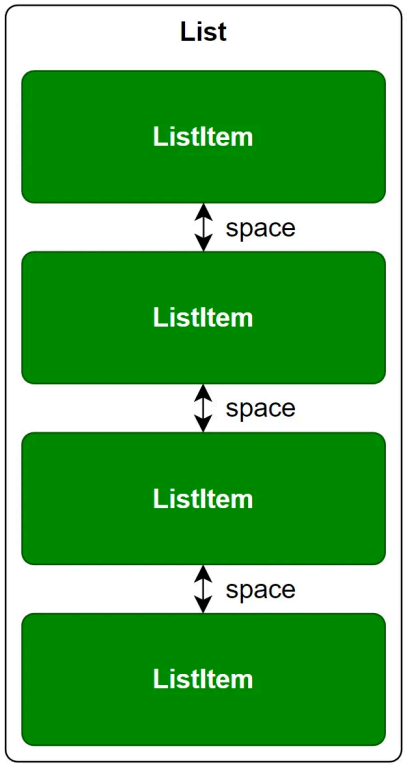
List 组件的参数定义如下，下面逐一介绍每个参数

代码块

```
1   List(value?:{space?: number | string, scroller?: Scroller})
```

4.2.1 `space` 列表项间距

`space` 参数用于设置列表项的间距，如下图所示



4.2.2 `scroller` 列表滚动控制器

`scroller` 参数用于绑定列表滚动控制器（**Scroller**），**Scroller**可以控制列表的滚动，例如列表返回顶部功能



代码块

```
@Entry
@Component
struct ScrollerPage {
  data: number[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
  scroller: Scroller = new Scroller();

  build() {
    Stack({ alignContent: Alignment.BottomEnd }) {
      List({ space: 20, scroller: this.scroller }) {
        ForEach(this.data, (item:number) => {
          ListItem() {
            Text(item.toString())
              .itemTextStyle()
```

```
          }
        })
      }.listStyle()
      .height('100%')
      .width('100%')

      Button({ type: ButtonType.Circle }) {
        Image($r('app.media.icon_top'))
          .width(40)
          .height(40)
      }
      .width(60)
      .height(60)
      .backgroundColor(Color.Orange)
      .offset({ x: -20, y: -100 })
      .onClick(() => {
        this.scroller.scrollToIndex(0) // 实例化scroller内的方法scrollToIndex
      })
    }
  }
}

@Extend(Text) function itemTextStyle() {
  .height(80)
  .width('100%')
  .textAlign(TextAlign.Center)
  .fontColor(Color.White)
  .fontSize(40)
  .fontWeight(FontWeight.Bold)
  .backgroundColor('#008a00')
  .borderRadius(10)
}

@Extend(List) function listStyle() {
  .backgroundColor(Color.White)
  .padding(20)
```

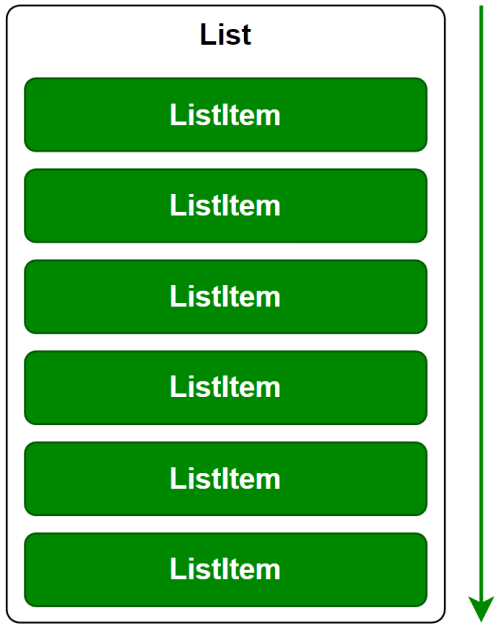
```
50   }
```

4.3 常用属性

4.3.1 主轴方向 `listDirection()`

使用 `listDirection()` 方法可以设置列表的主轴方向（即列表的排列和滚动方向），

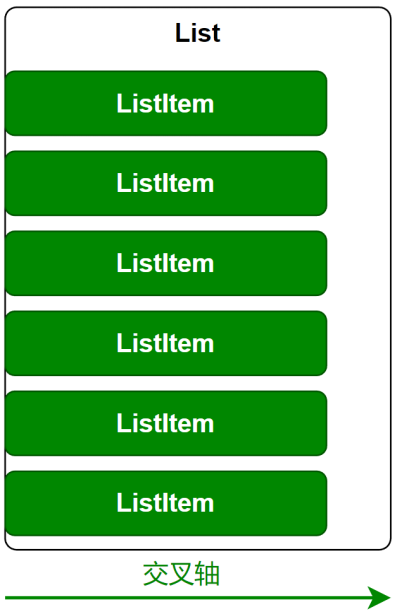
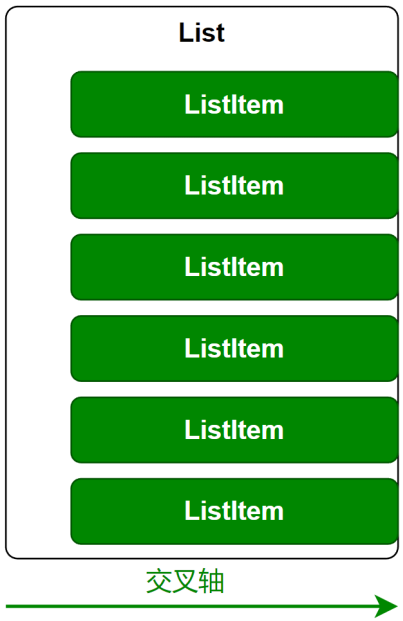
其参数类型为枚举类型 `Axis` ，可选的枚举值如下

名称	描述	效果
`Axis.Vertical`	纵向	
`Axis.Horizontal`	横向	



4.3.2 交叉轴对齐方式 `alignListItem()`

使用 `alignListItem()` 方法可以设置子组件在交叉轴方向的对齐方式，其参数类型为枚举类型 `ListItemAlign` ，可选的枚举值有

名称	`ListItemAlign.Start`	`ListItemAlign.Center`	`ListItemAlign.End`
描述	交叉轴起始端对齐	交叉轴居中对齐	交叉轴末端对齐
效果（以 `listDirection=Axis.Vertical` 为例）			

代码块

```
1  @Entry
2  @Component
```

```
struct AlignPage {
  data: number[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

  build() {

    List({ space: 20 }) {
      ForEach(this.data, (item) => {
        ListItem() {
          Text(item.toString())
            .height(80)
            .width(320)
            .itemTextStyle()
        }
      })
    }.listStyle()
    .height('100%')
    .width('100%')
    .alignListItem(ListItemAlign.Center)

  }
}

@Extend(Text) function itemTextStyle() {
  .textAlign(TextAlign.Center)
  .fontColor(Color.White)
  .fontSize(40)
  .fontWeight(FontWeight.Bold)
  .backgroundColor('#008a00')
  .borderRadius(10)
}

@Extend(List) function listStyle() {
  .backgroundColor(Color.White)
  .padding({ top: 20, bottom: 20 })
}
```

4.3.3 元素分割线 `divider()`

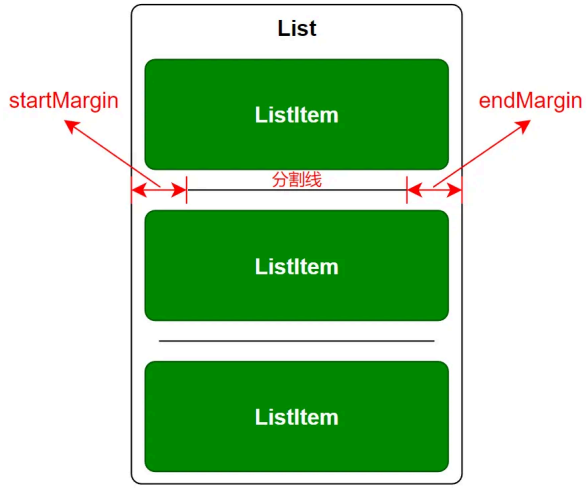
使用 `divider()` 属性可设置列表元素分割线样式，该方法的参数定义如下

代码块

```
divider(value: {strokeWidth: Length, color?: ResourceColor, startMargin?: Length, endMargin?: Length})
```

各参数的含义如下

参数	含义
strokeWidth	分割线线宽
color	分割线颜色
startMargin	分割线起始端到列表侧边距离（如下图所示）
endMargin	分割线末端到列表侧边距离（如下图所示）



代码块

```
@Entry
@Component
struct DividerPage {
  data: number[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

  build() {

    List({ space: 20 }) {
      ForEach(this.data, (item:number) => {
        ListItem() {
          Text(item.toString())
            .height(80)
            .width(320)
            .itemTextStyle()
        }
      })
    }
    .listStyle()
    .height('100%')
    .width('100%')
    .alignListItem(ListItemAlign.Center)
    .divider({
      strokeWidth: 1,
      color: Color.Orange,
      startMargin: 30,
      endMargin: 30
    })

  }
}

@Extend(Text) function itemTextStyle() {
  .textAlign(TextAlign.Center)
  .fontColor(Color.White)
  .fontSize(40)
```

```
    .fontWeight(FontWeight.Bold)
    .backgroundColor('#008a00')
    .borderRadius(10)
  }

  @Extend(List) function listStyle() {
    .backgroundColor(Color.White)
    .padding({ top: 20, bottom: 20 })
  }
```

4.3.4 滚动条样式 `scrollBar()`

使用 `scrollBar()` 方法可以设置**滚动条状态**，该方法的参数类型为枚举类型 `BarState` ，可选的枚举值如下

名称	描述
BarState.Off	不显示
BarState.On	常驻显示
BarState.Auto	按需显示(触摸时显示，2s后消失)

代码块

```
@Entry
@Component
struct ScrollBarPage {
  data: number[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

  build() {

    List({ space: 20 }) {
      ForEach(this.data, (item:string) => {
        ListItem() {
          Text(item.toString())
```

```
            .height(80)
            .width(320)
            .itemTextStyle()
        }
      })
    }.listStyle()
    .height('100%')
    .width('100%')
    .alignListItem(ListItemAlign.Center)
    .scrollBar(BarState.Auto)

  }
}

@Extend(Text) function itemTextStyle() {
  .textAlign(TextAlign.Center)
  .fontColor(Color.White)
  .fontSize(40)
  .fontWeight(FontWeight.Bold)
  .backgroundColor('#008a00')
  .borderRadius(10)
}

@Extend(List) function listStyle() {
  .backgroundColor(Color.White)
  .padding({ top: 20, bottom: 20 })
}
```

4.3.5 粘性标题 `stick`

通过 `stick` 实现了Header吸顶和Footer吸底的效果。

配合ListItemGroup组件使用，设置ListItemGroup中header和footer是否要吸顶或吸底。

sticky属性可以设置为 `StickyStyle.Header` | `StickyStyle.Footer` 以同时支持header吸顶和footer吸底。



代码块

```
interface TimeTable {
  title: string,
  projects: string[]
}

@Entry
@Component
struct ListItemGroupExample {
  @State timeTable: TimeTable[] = [
    {
      title: '星期一',
      projects: ['语文', '数学', '英语']
    },
    {
      title: '星期二',
      projects: ['物理', '化学', '生物']
    },
```

```
    {
      title: '星期三',
      projects: ['历史', '地理', '政治']
    },
    {
      title: '星期四',
      projects: ['美术', '音乐', '体育']
    }
  ];

  @Builder
  itemHead(text: string) {
    Text(text)
      .fontSize(20)
      .backgroundColor(0xAABBCC)
      .width('100%')
      .padding(10)
  }

  @Builder
  itemFoot(num: number) {
    Text('共' + num + '节课')
      .fontSize(16)
      .backgroundColor(0xAABBCC)
      .width('100%')
      .padding(5)
  }

  build() {
    Column() {
      List({ space: 20 }) {
        ForEach(this.timeTable, (item: TimeTable) => {
          ListItemGroup({ header: this.itemHead(item.title),
           footer: this.itemFoot(item.projects.length) }) {
            ForEach(item.projects, (project:string) => {
              ListItem() {
                Text(project)
```

```
                    .width('100%')
                    .height(100)
                    .fontSize(20)
                    .textAlign(TextAlign.Center)
                    .backgroundColor(0xFFFFFF)
                }
              })
            }.divider({ strokeWidth: 1, color: Color.Blue }) // 每行之间的分界线
          })
        }
        .width('90%')
        .sticky(StickyStyle.Header | StickyStyle.Footer)
        .scrollBar(BarState.Off)
      }.width('100%').height('100%').backgroundColor(0xDCDCDC).padding({ top:
    5 })
    }
}
```

4.4 列表布局List 课后练习

4.4.1 需求效果

1.使用列表布局List组件实现通讯录,并添加吸顶效果。

2.添加分割线 divider()

3.点击“通讯录”回到顶部 scroller



4.4.2 代码实现

代码块

```
1
2  interface DataItem {
3    key: string;
4    items: string[];
5  }
6
7  @Entry
8  @Component
9  struct ScrollBarPage {
10   // 使用数组来存储数据
```

```
  @State data: DataItem[] = [
    { key: 'A', items: ['Angle', '安欣', '安静'] },
    { key: 'B', items: ['白娘子', '伯父'] },
    { key: 'C', items: ['蔡坤坤', '裁缝', '村长','春花','丑八怪'] },
  ];
@Builder Key(key:string){
  Text(key)
    .fontSize(30)
    .fontWeight(500)
    .textAlign(TextAlign.Start)
    .padding({ left: 20 })
    .backgroundColor('#f6f6f6')
    .width(320)
    .height(50)
}
  build() {
    Column() {
      Text('通讯录')
      .fontSize(40)
      .fontWeight(FontWeight.Bolder)
      .textAlign(TextAlign.Start)
      .width('100%')
      .padding({left:30})
      List({ space: 20 }) {
        ForEach(this.data, (group: DataItem) => {
          ListItemGroup({header:this.Key(group.key)}) {
            ForEach(group.items, (item: string) => {
              ListItem() {
                Row() {
                  Image($r('app.media.img_user_avatar')).width(50).height(50)
                  Text(item.toString())
                    .height(80)
                    .width(320)
                    .margin({ left: 30 })
                    .fontSize(20)
                }
              }.width(320).padding({ left: 10 })
```

```
            })
          }
        })
      }
      .sticky(StickyStyle.Header)
      .listStyle()
      .height('100%')
      .width('100%')
      .alignListItem(ListItemAlign.Center)
      .scrollBar(BarState.Auto)
    }
  }
}

@Extend(List)
function listStyle() {
  .backgroundColor(Color.White)
  .padding({ top: 20, bottom: 20 })
}
```

5. 轮播图 （Swiper）

5.1 概述

Swiper组件提供滑动轮播显示的能力。

Swiper本身是一个容器组件，当设置了多个子组件后，可以对这些子组件进行轮播显示。

通常，在一些应用首页显示推荐的内容时，需要用到轮播显示的能力。



5.2 语法

1. 轮播内容：内容作为Swiper的子组件即可
2. 尺寸：
   a. 设置 Swiper 的尺寸：内容会拉伸为和 Swiper 一致（**优先级高**）
   b. 设置内容尺寸：会将Swiper撑开

```
Swiper() {
  // 轮播内容
  // (设置尺寸，撑开swiper)
}
// 设置尺寸（内容拉伸、优先级高）
.width('100%')
.height(100)
```

3. 基本使用



代码块

```
@Entry
@Component
struct SwiperPage {
  // Swiper 基本使用
  build() {
    Column() {
      Text('Swiper基本使用')
        .fontSize(20)
        .fontWeight(900)
        .padding(10)
      Swiper() {
        Text('0')
          .textAlign(TextAlign.Center)
          .backgroundColor(Color.Red)
          .fontColor(Color.White)
          .fontSize(30)
```

```
17          Text('1')
18            .textAlign(TextAlign.Center)
19            .backgroundColor(Color.Green)
20            .fontColor(Color.White)
21            .fontSize(30)
22          Text('2')
23            .textAlign(TextAlign.Center)
24            .backgroundColor(Color.Blue)
25            .fontColor(Color.White)
26            .fontSize(30)
27        }
28        .width('100%')
29        .height(100)
30      }
31      .width('100%')
32      .height('100%')
33    }
34  }
```

5.3 常用属性

5.3.1 自动播放&循环播放

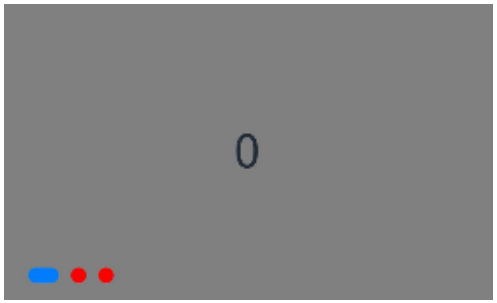
属性	参数类型	说明
autoPlay	boolean	子组件是否自动播放。默认值：false **说明**：loop为false时，自动轮播到最后一页时停止轮播。 手势切换后不是最后一页时继续播放。
interval	number	使用自动播放时播放的时间间隔，单位为毫秒。默认值：3000
vertical	boolean	是否为纵向滑动。默认值：false
loop	boolean	是否开启无限循环播放。 设置为true时表示无限循环播放，设置为false时表示只播放一次。 默认值：true

5.3.2 导航点样式

Swiper提供了默认的导航圆点样式和导航箭头样式，导航点默认显示在Swiper下方居中位置，

通过indicator属性，开发者可以设置导航点相对于Swiper组件上下左右四个方位的位置，同时也可以设置每个导航点的尺寸、颜色、蒙层和被选中导航点的颜色。

5.3.2.1 导航圆点样式



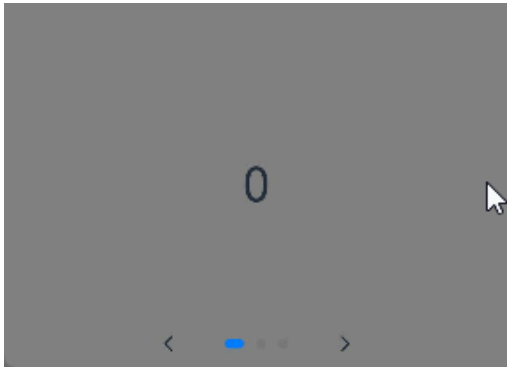
代码块

```
// 导航点直径设为30vp，左边距为0，导航点颜色设为红色。
Swiper() {
  // ...
}
.indicator(
  Indicator.dot()
    .left(0)
    .itemWidth(15)
    .itemHeight(15)
    .selectedItemWidth(30)
    .selectedItemHeight(15)
    .color(Color.Red)
    .selectedColor(Color.Blue)
)
```

5.3.2.2 导航箭头样式

Swiper通过设置displayArrow属性，可以控制导航点箭头的大小、位置、颜色，底板的大小及颜色，以及鼠标悬停时是否显示箭头。

- 箭头使用默认样式



代码块

```
Swiper() {
  // ...
}
.displayArrow(true, false)

```

参数名	类型	必填	说明
value	ArrowStyle | boolean	是	支持设置箭头和底板样式，异常场景使用ArrowStyle对象中的默认值。设置为false不显示箭头和底板，true显示默认的箭头和底板样式。默认值：false
isHoverShow	boolean	否	设置鼠标悬停时是否显示箭头。默认值：false **说明：** 1、isHoverShow为false时，常驻显示箭头。 2、isHoverShow为true时，有导航点时鼠标悬停在导航点和箭头范围内显示箭头，无导航点时鼠标悬停在Swiper显示范围内显示箭头。 3、箭头显示时，支持点击翻页。

- 自定义箭头样式



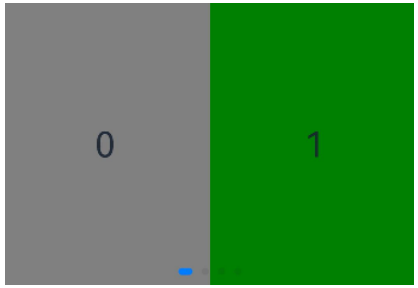
代码块

```
// 箭头显示在组件两侧，大小为18vp，导航点箭头颜色设为蓝色。
Swiper() {
  // ...
}
.displayArrow({
  showBackground: true,
  isSidebarMiddle: true,
  backgroundSize: 24,
  backgroundColor: Color.White,
  arrowSize: 18,
  arrowColor: Color.Blue
  }, false)

```

5.3.3 每页显示多个子页面

Swiper支持在一个页面内同时显示多个子组件，通过`displayCount`属性设置。



代码块

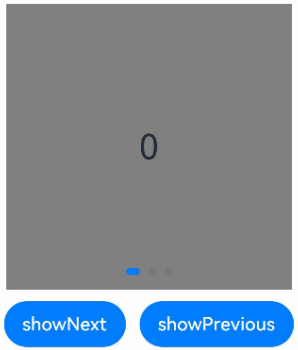
```
Swiper() {
  Text('0')
    .width(250)
    .height(250)
    .backgroundColor(Color.Gray)
    .textAlign(TextAlign.Center)
    .fontSize(30)
  Text('1')
    .width(250)
    .height(250)
    .backgroundColor(Color.Green)
    .textAlign(TextAlign.Center)
    .fontSize(30)
  Text('2')
    .width(250)
    .height(250)
    .backgroundColor(Color.Pink)
    .textAlign(TextAlign.Center)
    .fontSize(30)
  Text('3')
    .width(250)
    .height(250)
    .backgroundColor(Color.Blue)
    .textAlign(TextAlign.Center)
    .fontSize(30)
}
.indicator(true)
```

```
28   .displayCount(2)
29
```

5.3.4 页面切换方式

Swiper支持手指滑动、点击导航点和通过控制器三种方式切换页面，以下示例展示通过控制器切换页面的方法。

通过 `SwiperController` 控制器的 `showNext()` 和 `showPrevious()` 方法，切换页面



代码块

```
1   @Entry
2   @Component
3   struct SwiperDemo {
4     //  创建轮播图的实例对象swiperController
5     private swiperController: SwiperController = new SwiperController();
6
7     build() {
8       Column({ space: 5 }) {
9         Swiper(this.swiperController) {
10          Text('0')
11            .width(250)
12            .height(250)
13            .backgroundColor(Color.Gray)
14            .textAlign(TextAlign.Center)
```

```
15            .fontSize(30)
16          Text('1')
17            .width(250)
18            .height(250)
19            .backgroundColor(Color.Green)
20            .textAlign(TextAlign.Center)
21            .fontSize(30)
22          Text('2')
23            .width(250)
24            .height(250)
25            .backgroundColor(Color.Pink)
26            .textAlign(TextAlign.Center)
27            .fontSize(30)
28        }
29        .indicator(true)
30
31        Row({ space: 12 }) {
32          Button('showNext')
33            .onClick(() => {
34              this.swiperController.showNext(); // 通过controller切换到后一页
35            })
36          Button('showPrevious')
37            .onClick(() => {
38              this.swiperController.showPrevious(); // 通过controller切换到前一页
39            })
40        }.margin(5)
41      }.width('100%')
42      .margin({ top: 5 })
43    }
44  }
45
```

5.4 练习

轮播图效果:



1. 宽高：100%、150
2. 循环、自动轮播，间隔 2000
3. 圆点指示器：
   a. 选中颜色：白色
   b. 选中宽高：30、4
   c. 默认宽高：10、4
4. 箭头指示器：
   a. 背景色：默认灰色
   b. 箭头色：白色
   c. 背景大小：24
   d. 箭头大小：18

SwiperTest.ets

```
@Entry
@Component
struct SwiperTest {
  build() {
    Column() {
      Text('Swiper案例-小米')
        .fontSize(20)
```

```
          .fontWeight(900)
          .padding(10)
        Swiper() {
          Image($r('app.media.swiper3')).ImageExtendStyle()
          Image($r('app.media.swiper2')).ImageExtendStyle()
          Image($r('app.media.swiper1')).ImageExtendStyle()
          Image($r('app.media.swiper4')).ImageExtendStyle()
        }.autoPlay(true)
        .interval(2000)
        .indicator(
          Indicator.dot()
            .selectedItemWidth(30)
            .itemWidth(10)
            .selectedColor(Color.White)
        )
        .displayArrow({
          showBackground: true,
          isSidebarMiddle: true,
          backgroundSize: 24,
          arrowSize: 18,
          arrowColor: Color.White
        }, false)
      }
      .width('100%')
      .height('100%')
    }
  }
  @Extend(Image) function ImageExtendStyle(){
    .width('100%')
    .height(150)
  }
```

6. 层叠布局（Stack）

6.1 概述

层叠布局是指将多个组件沿垂直于屏幕的方向堆叠在一起，类似于图层的叠加。以下效果都可以通过层叠布局实现



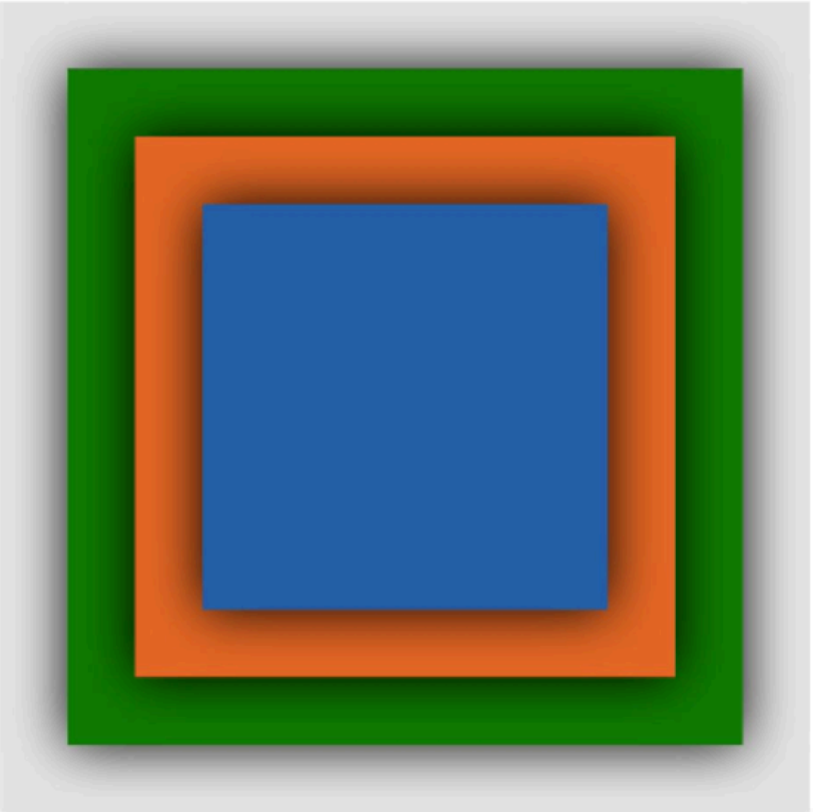
层叠布局可通过**Stack**容器组件实现，其子元素会按照其添加顺序依次叠加在一起，后添加的子元素位于先添加的子元素之上。具体效果如下

代码	效果

代码块

```
Stack() {
  Row()
    .width(250)
    .height(250)
    .backgroundColor('#107B02') //绿色
    .shadow({radius:50})
```

```
    Row()
      .width(200)
      .height(200)
      .backgroundColor('#E66826') //橙色
      .shadow({radius:50})
    Row()
      .width(150)
      .height(150)
      .backgroundColor('#255FA7') //蓝色
      .shadow({radius:50})
  }
  .width(300)
  .height(300)
  .backgroundColor('#E5E5E5') //灰色
```



代码块

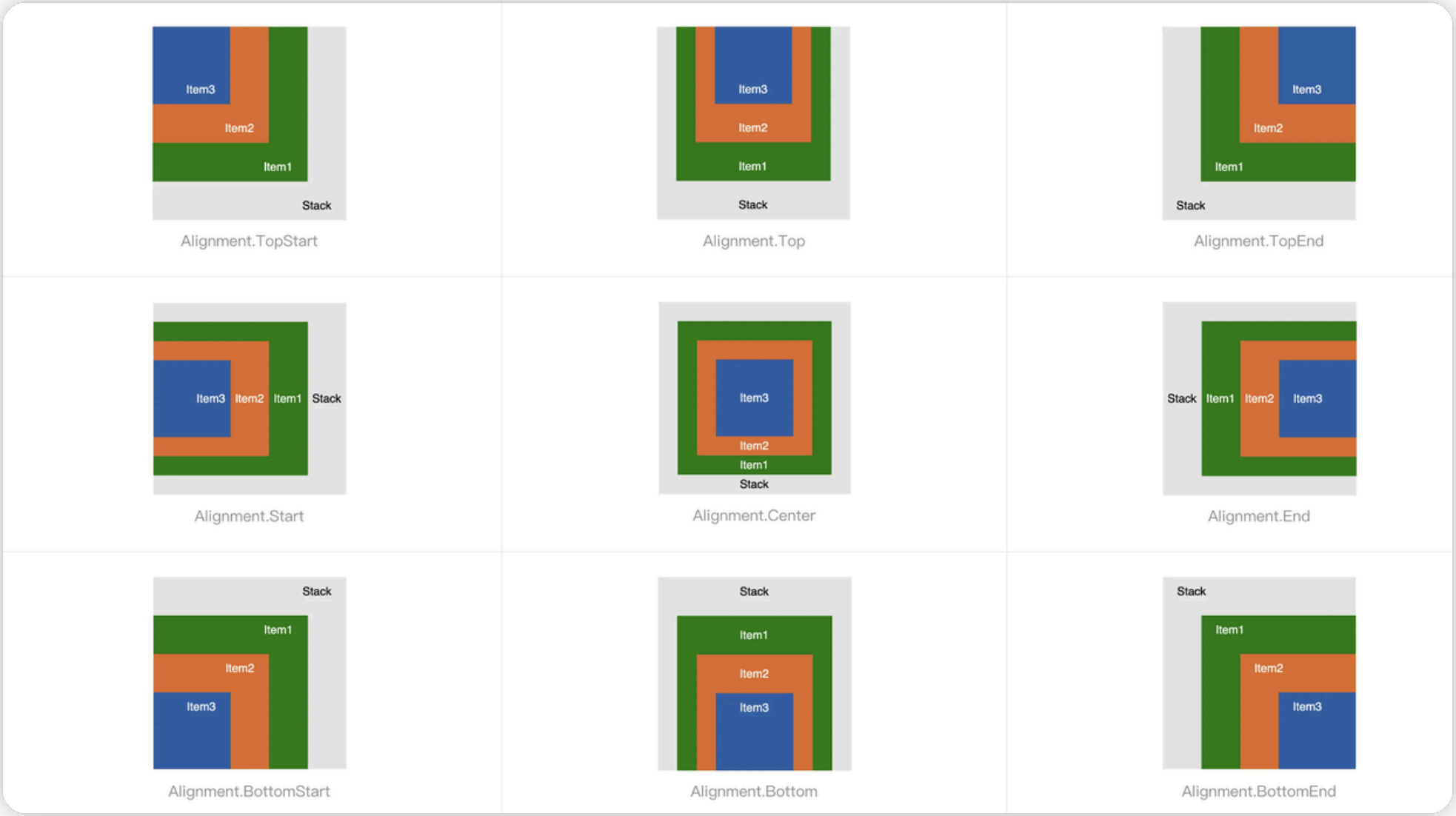
```
@Entry
@Component
struct StackPage {
  build() {
    Column() {
      Stack() {
        Row()
          .width(250)
          .height(250)
          .backgroundColor('#107B02') //绿色
          .shadow({radius:50})

```

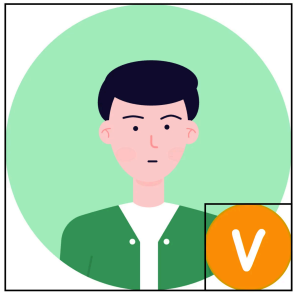
```
13          Row()
14            .width(200)
15            .height(200)
16            .backgroundColor('#E66826') //橙色
17            .shadow({radius:50})
18          Row()
19            .width(150)
20            .height(150)
21            .backgroundColor('#255FA7') //蓝色
22            .shadow({radius:50})
23        }
24        .width(300)
25        .height(300)
26        .backgroundColor('#E5E5E5')
27
28      }.width('100%')
29      .height('100%')
30      .justifyContent(FlexAlign.Center)
31    }
32  }
```

6.2 参数

Stack组件的参数类型为 `{ alignContent?: Alignment }` ， `alignContent` 用于设置子组件的对齐方式，该属性可通过枚举类型 `Alignment` 进行设置，可选的枚举值及其效果如下图所示



该参数的一个实际使用场景如下：



代码块

```
@Entry
@Component
struct AlignmentContentPage {
  build() {
    Column() {
      Stack({alignContent:Alignment.BottomEnd}) {
        Image($r('app.media.img_avatar'))
          .width('100%')
          .height('100%')
        Image($r('app.media.icon_v'))
          .width(60)
          .height(60)
      }
      .width(200)
      .height(200)

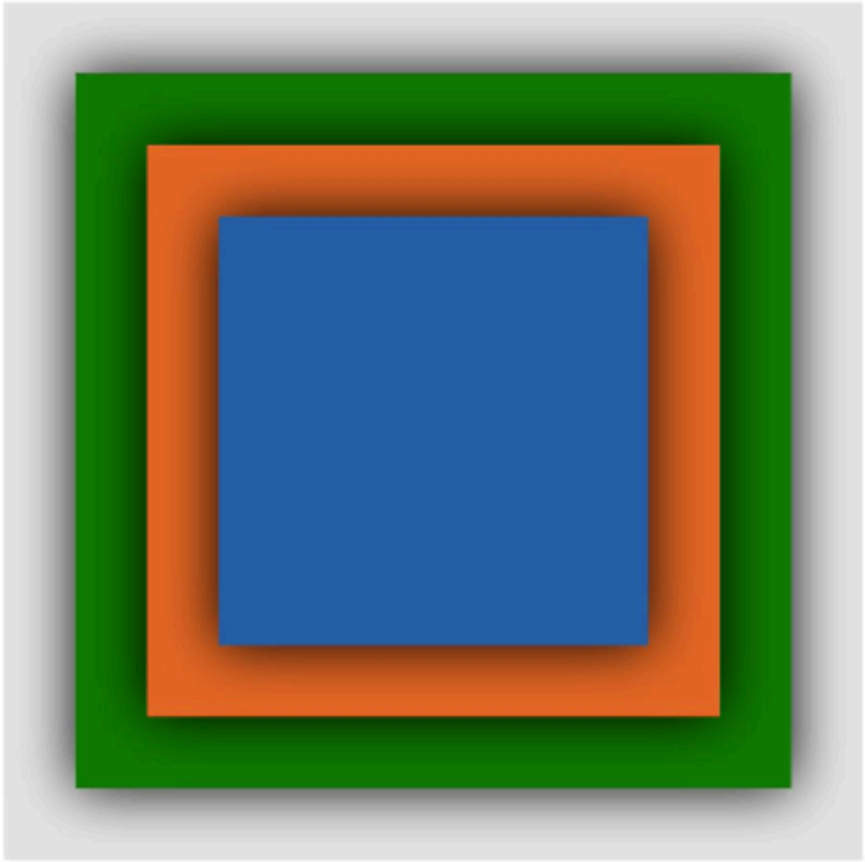
    }.width('100%')
    .height('100%')
    .justifyContent(FlexAlign.Center)
  }
}
```

6.3 实用技巧

6.3.1 子组件Z轴控制

Stack容器中子组件的层级除了可按照添加顺序决定，还能通过 `zIndex()` 进行手动的设置， `zIndex` 的值越大，层级越高。

代码	效果

代码块
1 Stack() {
2 Row()
3 .width(150)
4 .height(150)
5 .backgroundColor('#255FA7') //蓝色
6 .shadow({ radius: 50 })
7 .zIndex(3)
8
9 Row()
10 .width(200)
11 .height(200)
12 .backgroundColor('#E66826') //橙色
13 .shadow({ radius: 50 })
14 .zIndex(2)
15
16 Row()
17 .width(250)
18 .height(250)
19 .backgroundColor('#107B02') //绿色
20 .shadow({ radius: 50 })
21 .zIndex(1)
22
23 }.width(300)
24 .height(300)
25 .backgroundColor('#E5E5E5') //灰色
	

代码块

```
@Entry
@Component
struct ZIndexPage {
  build() {
    Column() {
      Stack() {
        Row()
          .width(150)
          .height(150)
          .backgroundColor('#255FA7') //蓝色
          .shadow({ radius: 50 })
          .zIndex(3)

        Row()
          .width(200)
          .height(200)
          .backgroundColor('#E66826') //橙色
          .shadow({ radius: 50 })
          .zIndex(2)

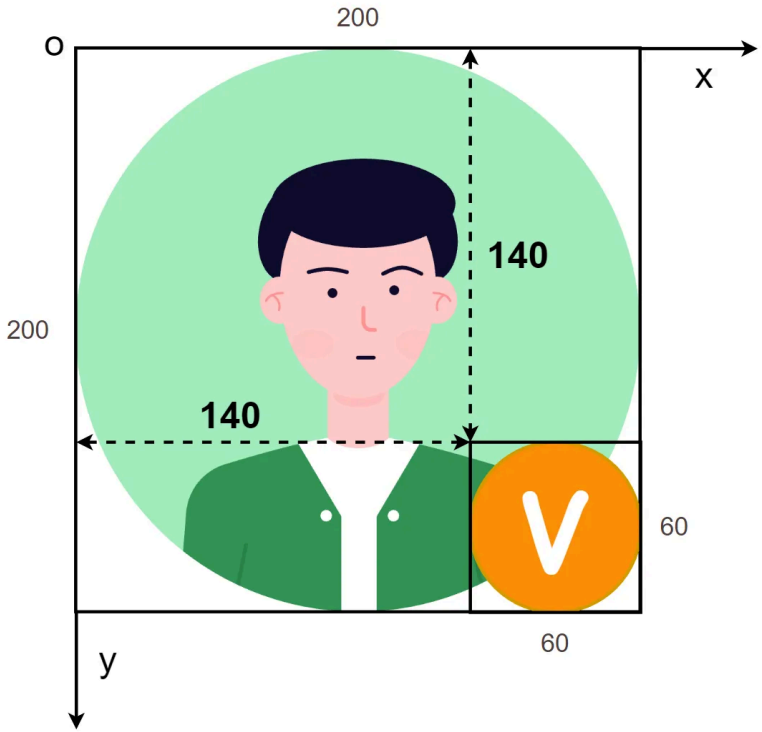
        Row()
          .width(250)
          .height(250)
          .backgroundColor('#107B02') //绿色
          .shadow({ radius: 50 })
          .zIndex(1)

      }.width(300)
      .height(300)
      .backgroundColor('#E5E5E5')

    }.width('100%')
    .height('100%')
    .justifyContent(FlexAlign.Center)
  }
}
```

6.3.2 子组件精确定位 `position()`

Stack容器的子组件可使用 `position()` 方法进行更精确的定位，该方法可设置子组件**左上角**相对于**Stack**容器**左上角**的偏移量，具体效果如下

代码	效果
代码块 1 Stack() { 2 Image($r('app.media.img_avatar')) 3 .width('100%') 4 .height('100%') 5 Image($r('app.media.icon_v')) 6 .width(60) 7 .height(60) 8 .position({ x: 140, y: 140 }) 9 } 10 .width(200) 11 .height(200)	

代码块

```
1  @Entry
2  @Component
3  struct PositionPage {
```

```
  build() {
    Column() {
      Stack() {
        Image($r('app.media.img_avatar'))
          .width('100%')
          .height('100%')
        Image($r('app.media.icon_v'))
          .width(60)
          .height(60)
          .position({ x: 140, y: 140 })
      }
      .width(200)
      .height(200)

    }.width('100%')
    .height('100%')
    .justifyContent(FlexAlign.Center)
  }
}
```